Java

Introduction

- Java is a high-level, object-oriented programming language developed by Sun Microsystems in 1995.
- It is platform-independent, meaning code can run anywhere with a Java Virtual Machine (JVM).
- Java is popular for its simplicity, security, and robustness, making it ideal for various applications.
- It supports object-oriented principles like inheritance, encapsulation, and polymorphism.
- Java’s "Write Once, Run Anywhere" philosophy allows for cross-platform compatibility.
- Java is widely used in web development, mobile apps, and large-scale enterprise systems.
- The language has a vast ecosystem, including extensive libraries, frameworks, and development tools.

History

- Java was invented by James Gosling and his team at Sun Microsystems in 1991 and named as ‘Oak’.
- In 1995, Oak was renamed as "Java“. Java is an island in Indonesia where the first coffee was produced.
- In 1995, Time magazine called Java one of the Ten Best Products of 1995.
- JDK 1.0 was released on January 23, 1996.
- In 2010, Oracle Corporation acquired Sun Microsystems and took over the development of Java.
- Java has since evolved with regular updates, becoming one of the most widely used programming languages globally.

Java Features

- Platform-Independent
- Object-Oriented
- Simple and Easy to Learn
- Secure
- Robust
- Multithreaded
- High Performance
- Rich Standard Library

Applications

- Desktop Applications such as acrobat reader, media player, antivirus, etc.
- Web Applications such as irctc.co.in, javatpoint.com, etc.
- Enterprise Applications such as banking applications.
- Mobile
- Embedded System
- Smart Card
- Robotics
- Games, etc.

Type of Java Applications

- **Standalone Application**
- **Web Application** : Applications that run on the server side to create dynamic web pages.
  Currently, Servlet, JSP, Struts, Spring, Hibernate, JSF, etc. technologies are used for creating web applications in Java.
- **Enterprise Application** : An application that is distributed in nature, such as banking applications, etc. is called an enterprise application. It has advantages like high-level security, load balancing, and clustering. In Java, EJB (Enterprise Java Beans) is used for creating enterprise applications.
- **Mobile Application**

Java Platforms / Editions

1) **Java SE (Java Standard Edition)** : It is a Java programming platform. It includes Java programming APIs such as java.lang, java.io, java.net, java.util, java.sql, java.math etc. It includes core topics like OOPs, String, Regex, Exception, Inner classes, Multithreading, I/O Stream, Networking, AWT, Swing, Reflection, Collection, etc.

2) **Java EE (Java Enterprise Edition)** : It is an enterprise platform that is mainly used to develop web and enterprise applications. It is built on top of the Java SE platform. It includes topics like Servlet, JSP, Web Services, EJB, JPA, etc.

3) **Java ME (Java Micro Edition)** : It is a micro platform that is dedicated to mobile applications.

4) **JavaFX** : It is used to develop rich internet applications. It uses a lightweight user interface API.

C
vs
C++
vs
Java

Metrics	C	C++	Java
Programming Paradigm	Procedural language	Object-Oriented Programming (OOP)	Pure Object Oriented
Origin	Based on assembly language	Based on C language	Based on C and C++
Developer	Dennis Ritchie in 1972	Bjarne Stroustrup in 1979	James Gosling in 1991
Translator	Compiler only	Compiler only	Interpreted language (Compiler + interpreter)
Platform Dependency	Platform Dependent	Platform Dependent	Platform Independent
Code execution	Direct	Direct	Executed by JVM (Java Virtual Machine)
Approach	Top-down approach	Bottom-up approach	Bottom-up approach
File generation	.exe files	.exe files	.class files
Pre-processor directives	Support header files (#include, #define)	Supported (#header, #define)	Use Packages (import)
keywords	Support 32 keywords	Supports 63 keywords	50 defined keywords
Datatypes (union, structure)	Supported	Supported	Not supported
Inheritance	No inheritance	Supported	Supported except Multiple inheritance
Overloading	No overloading	Support Function overloading (Polymorphism)	Operator overloading is not supported
Pointers	Supported	Supported	Not supported
Allocation	Use malloc, calloc	Use new, delete	Garbage collector
Exception Handling	Not supported	Supported	Supported
Templates	Not supported	Supported	Not supported
Destructors	No constructor neither destructor	Supported	Not supported
Multithreading/ Interfaces	Not supported	Not supported	Supported
Database connectivity	Not supported	Not supported	Supported
Storage Classes	Supported (auto, extern)	Supported (auto, extern)	Not supported

First Java Program

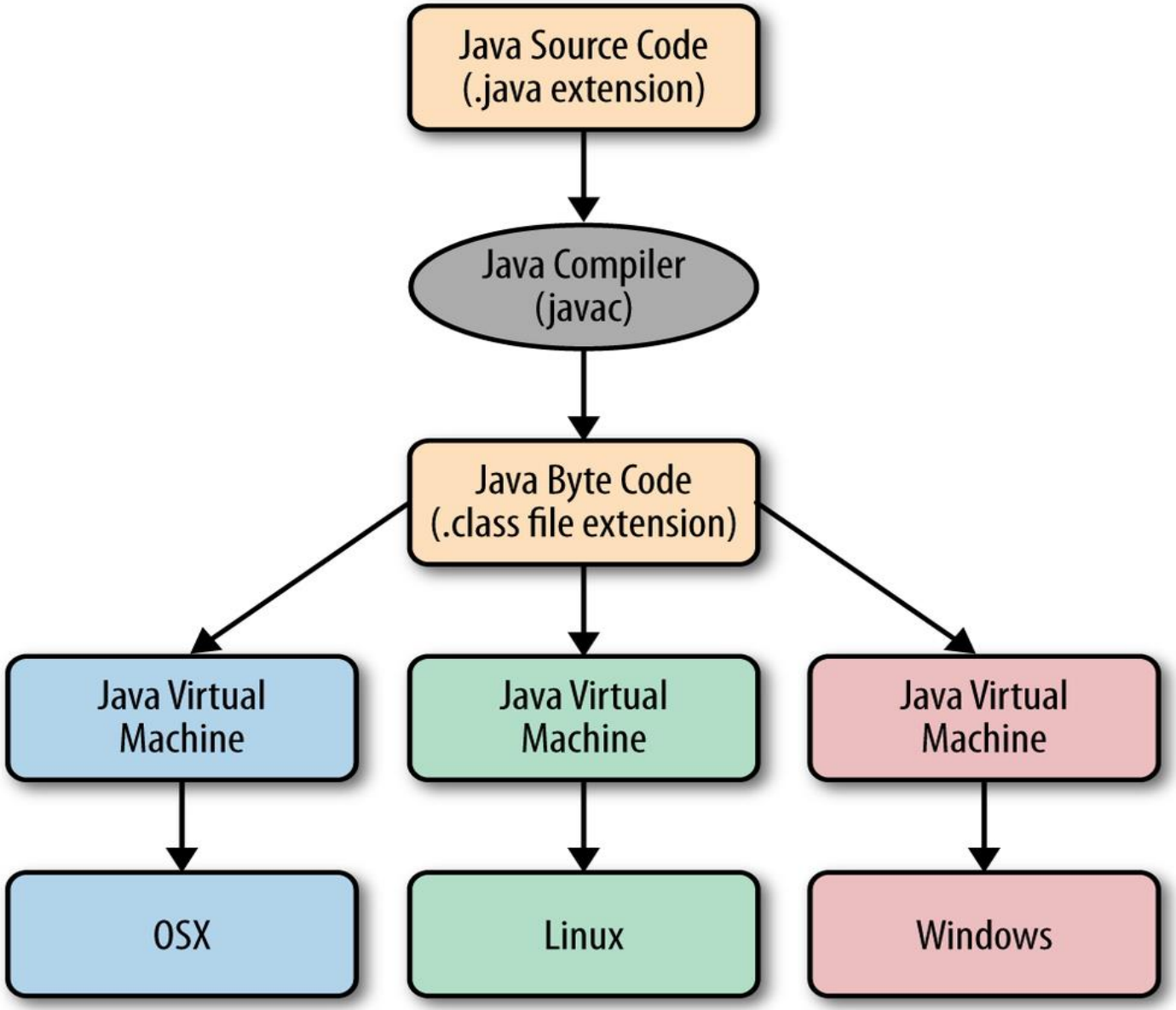
```
class Simple{
    public static void main(String args[]){
     System.out.println("Hello Java");
    }
}
```

To compile: javac Simple.java

To execute: java Simple

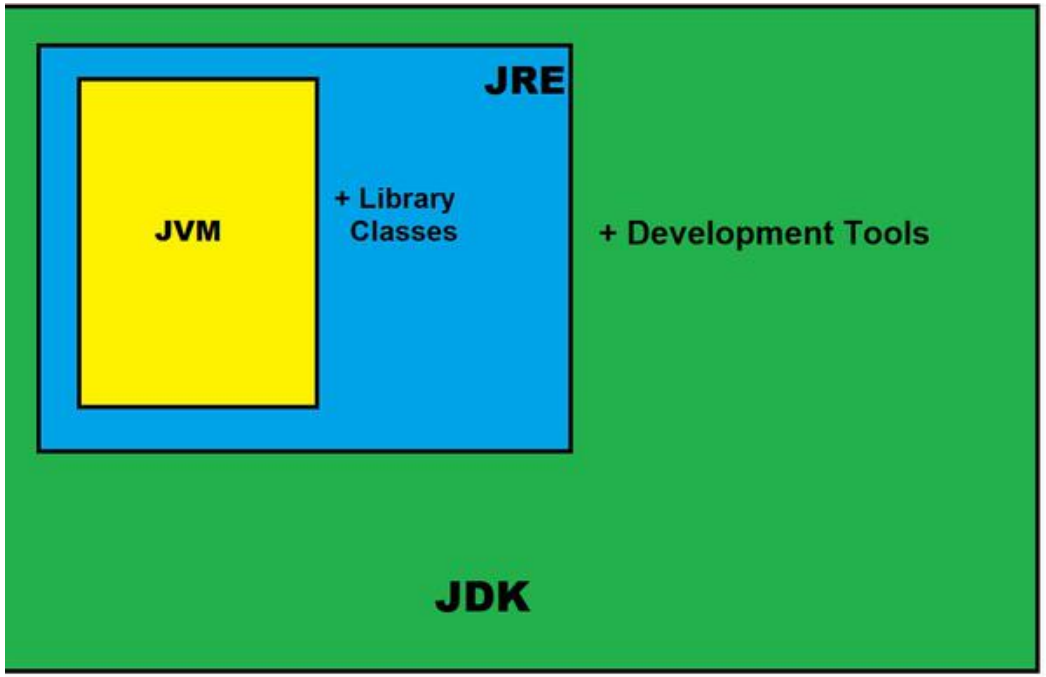
- System: A built-in class that provides access to system resources.
- out: A static member of the System class, which is an instance of PrintStream.
- println: A method of PrintStream that prints a line of text and then moves the cursor to the next line.

Java Compilation and Execution



Java Development Kit (JDK)

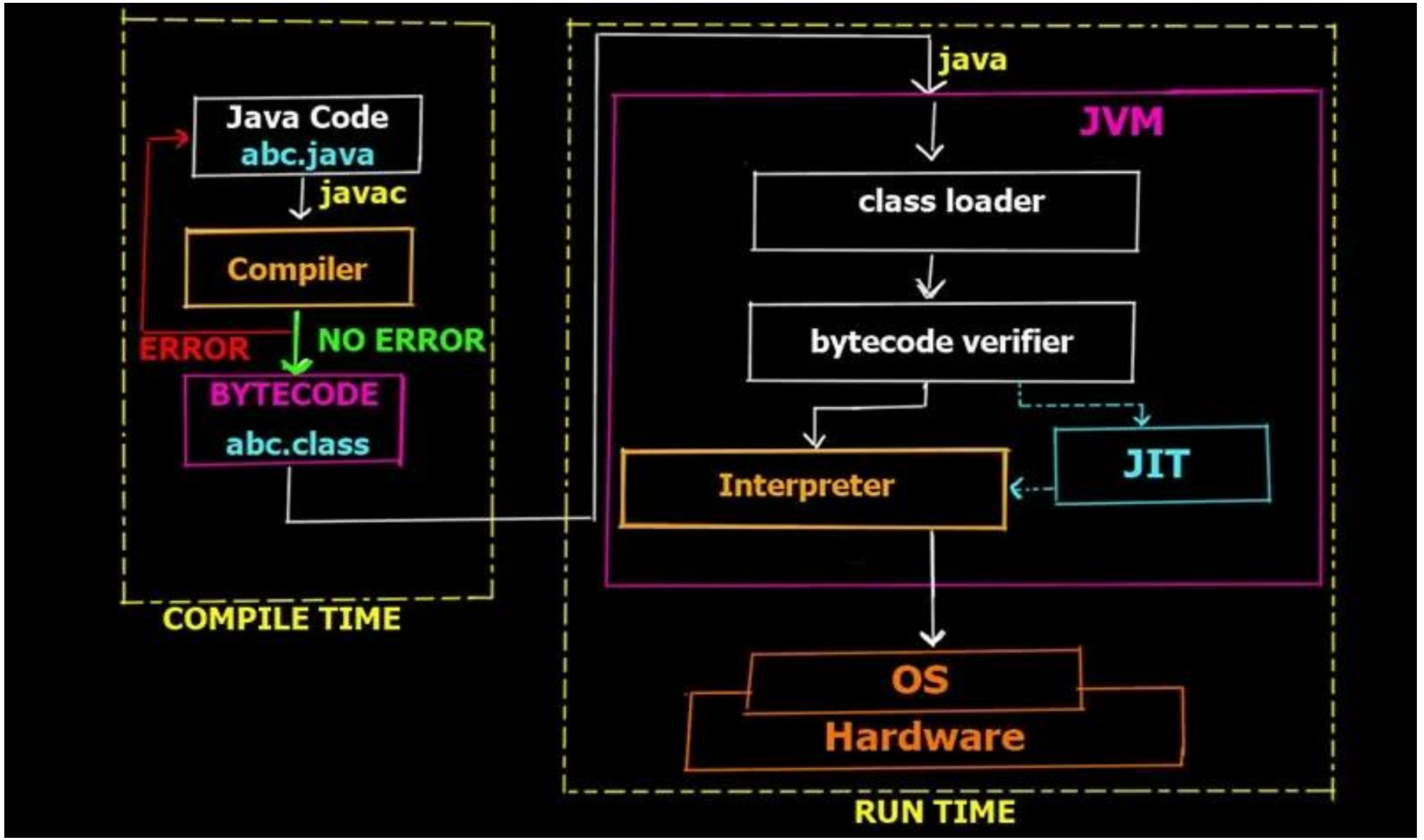
- The Java Development Kit (JDK) is a cross-platformed software development environment that offers a collection of tools and libraries necessary for developing Java-based software applications and applets.
- **JDK=JRE+Development Tools**
- **JDK contains:**
  - Java Runtime Environment (JRE),
  - An interpreter/loader (Java),
  - A compiler (javac),
  - An archiver (jar) and many more.



Java Virtual Machine

- JVM(Java Virtual Machine) acts as a run-time engine to run Java applications.
- JVM is the one that actually calls the **main** method present in a Java code.
- Java applications are called WORA (Write Once Run Anywhere). This is all possible because of JVM.
- JVMs are available for many hardware and software platforms.
- JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other.
- However, Java is platform independent.

Execution of Java Program



- **ClassLoader –**
  - **Loads .class files into memory** so the JVM can run them.
  - Java doesn’t load all classes at once — the ClassLoader loads them **when needed**.
  - It uses a **parent-first rule** (delegation) to avoid loading the same class multiple times.
  - You can create your **own ClassLoader** if you want to load classes in a special way (e.g., from a database or network).
- **Bytecode Verifier – Key Points**
  - **Checks the validity and safety of bytecode** before it is executed by the JVM.
  - Ensures **type safety**, correct stack usage, and that there are no illegal operations (like accessing private fields of another class).
  - **Prevents security vulnerabilities** like stack overflow, illegal casting, or memory corruption.
  - Acts as a **defensive mechanism** against malformed or malicious bytecode (important for running untrusted code).

- **Interpreter:**
  - Executes bytecode line-by-line at runtime.
  - Enables platform independence (JVM-based).
  - Allows quick program startup.
  - Supports dynamic features (e.g., reflection). This allows programmer to analyze and modify code at runtime.
  - Slower than native code; JIT used for optimization.

- **JIT (Just-In-Time) Compiler**
  - Converts **bytecode into native machine code** at runtime.
  - Helps Java run **faster** by avoiding repeated interpretation.
  - Only compiles **frequently used code**, so it's efficient.
  - Works **inside the JVM**, improving performance **without changing the code**.

Reasons Java Uses an Interpreter (at first)

1. Portability

Bytecode is platform-independent. An interpreter (JVM) can run the same bytecode on **any machine** without recompiling.

2. Quick Startup

Interpreting allows the program to start running immediately, without waiting to compile the whole code.

3. Dynamic Features

Java supports features like:

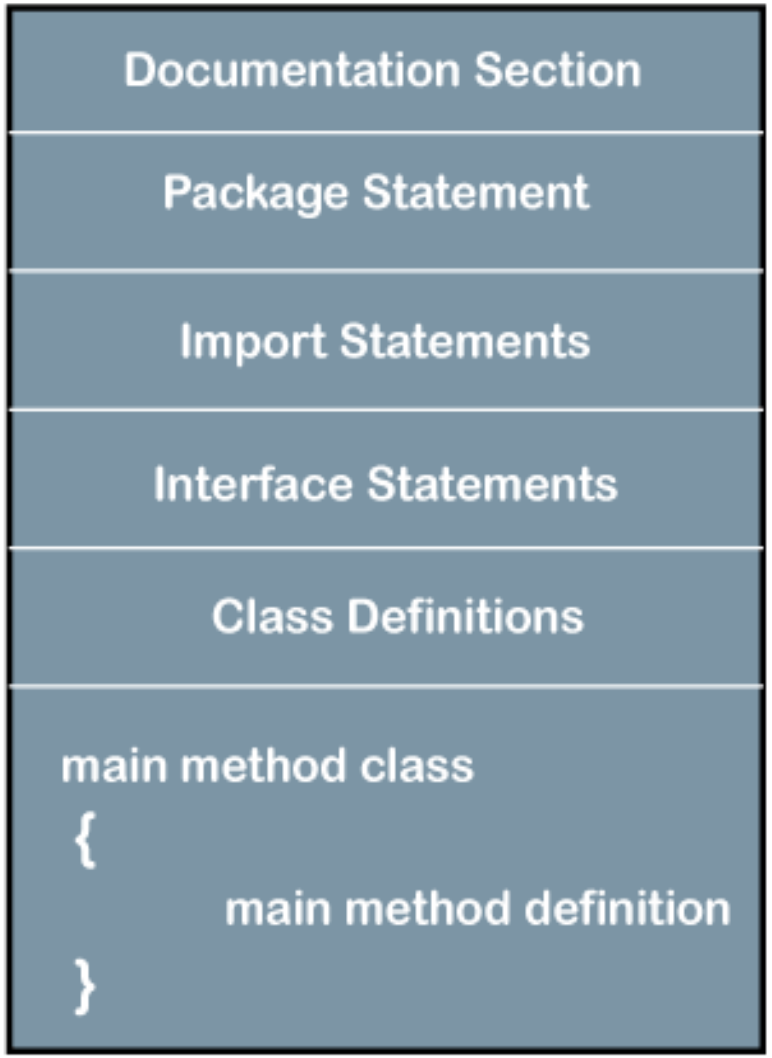
- Dynamic class loading,
- Reflection,
- Bytecode modification at runtime (e.g., for AOP or proxies).

These are easier to support in an interpreted environment than a statically compiled one.

How is java platform independent?

- The bytecode generated is a non-executable code and needs an interpreter to execute on a machine. This interpreter is the JVM and thus the Bytecode is executed by the JVM.
- Different platforms (Windows, Linux, macOS, etc.) have their own JVM implementations.
- The JVM abstracts the underlying operating system and hardware details, allowing Java programs to be executed in a consistent environment across different platforms.

Java Program structure



Structure of Java Program

Comments in java

- Single-line Comments
  - Single-line comments start with two forward slashes (//).
- Java Multi-line Comments
  - Multi-line comments start with /* and ends with */.

Java Program to compute square root

```
Import java.lang.Math;
public class SquareRoot {
    public static void main(String[] args) {
        double number = 16; // Example number
        double squareRoot = Math.sqrt(number);

        System.out.println("The square root of " + number + " is: " +
squareRoot);
    }
}
```

Java program with two classes

```
public class Rectangle {
    // Properties of the rectangle
    double length;
    double width;

    // Method to set the dimensions of the rectangle
    public void setDimensions(double l, double w) {
        length = l;
        width = w;
    }

    // Method to calculate the area of the rectangle
    public double calculateArea() {
        return length * width;
    }
}
```

```
public class RectangleAreaCalculator {
    public static void main(String[] args) {
        // Create a rectangle object
        Rectangle myRectangle = new Rectangle();

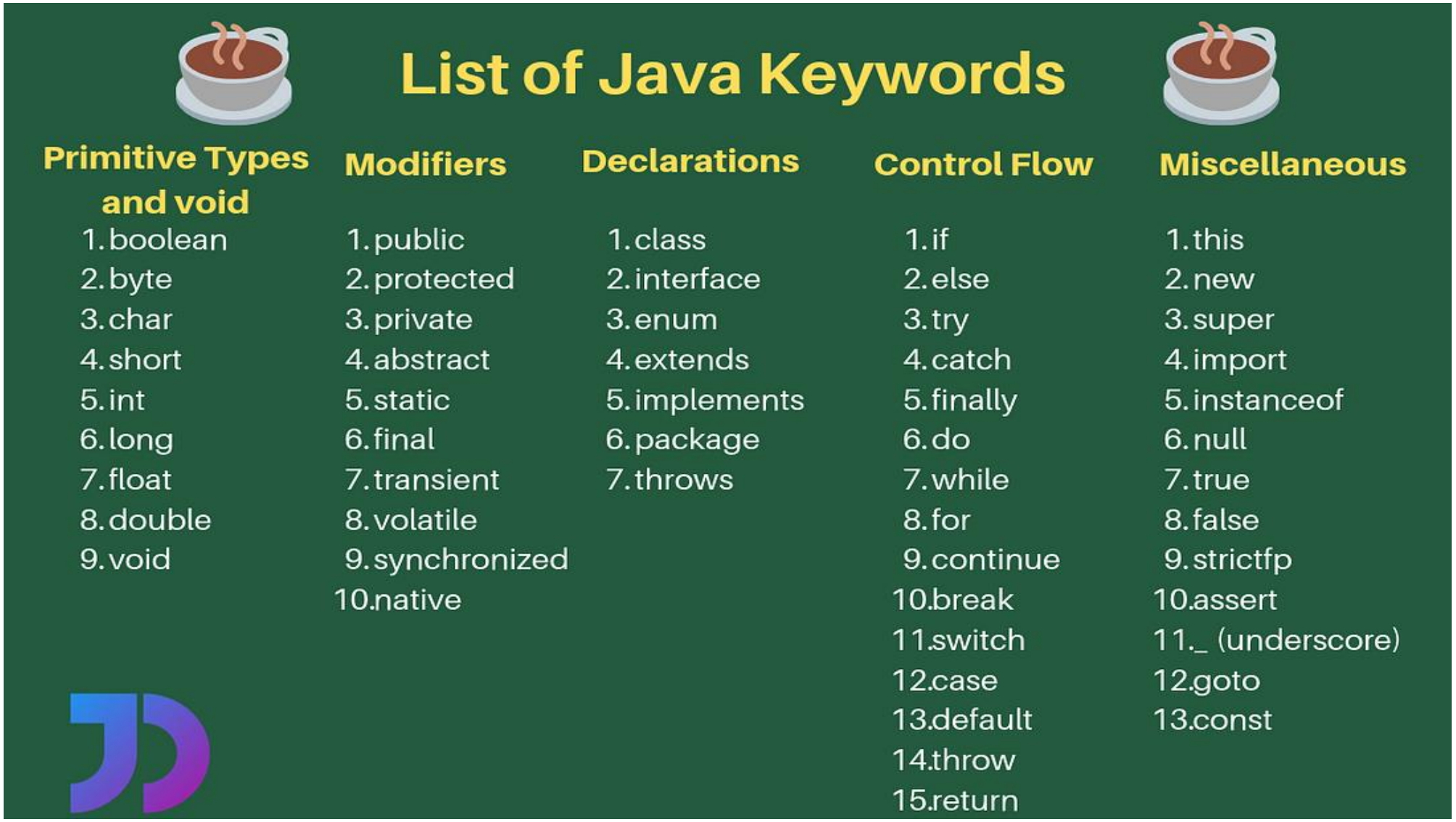
        // Set the dimensions of the rectangle
        myRectangle.setDimensions(5, 3);

        // Calculate and display the area of the rectangle
        double area = myRectangle.calculateArea();
        System.out.println("The area of the rectangle is: " +
area);
    }
}
```

Java Tokens

- **Tokens** are the smallest elements of a program that the Java compiler recognizes.
- They are the building blocks of a Java program, and every Java program is composed of a sequence of tokens.
- There are several types of tokens in Java:
  - Keywords: 50 keywords in java.
  - Identifiers
  - Literals : constants
  - Operators
  - Seperators : Parenthesis (), braces {}, brackets[], semicolon ; , comma, period

Keywords


List of Java Keywords
Primitive Types and void
1. boolean
2. byte
3. char
4. short
5. int
6. long
7. float
8. double
9. void
Modifiers
1. public
2. protected
3. private
4. abstract
5. static
6. final
7. transient
8. volatile
9. synchronized
10. native
Declarations
1. class
2. interface
3. enum
4. extends
5. implements
6. package
7. throws
Control Flow
1. if
2. else
3. try
4. catch
5. finally
6. do
7. while
8. for
9. continue
10. break
11. switch
12. case
13. default
14. throw
15. return
Miscellaneous
1. this
2. new
3. super
4. import
5. instanceof
6. null
7. true
8. false
9. strictfp
10. assert
11. _ (underscore)
12. goto
13. const

Identifiers

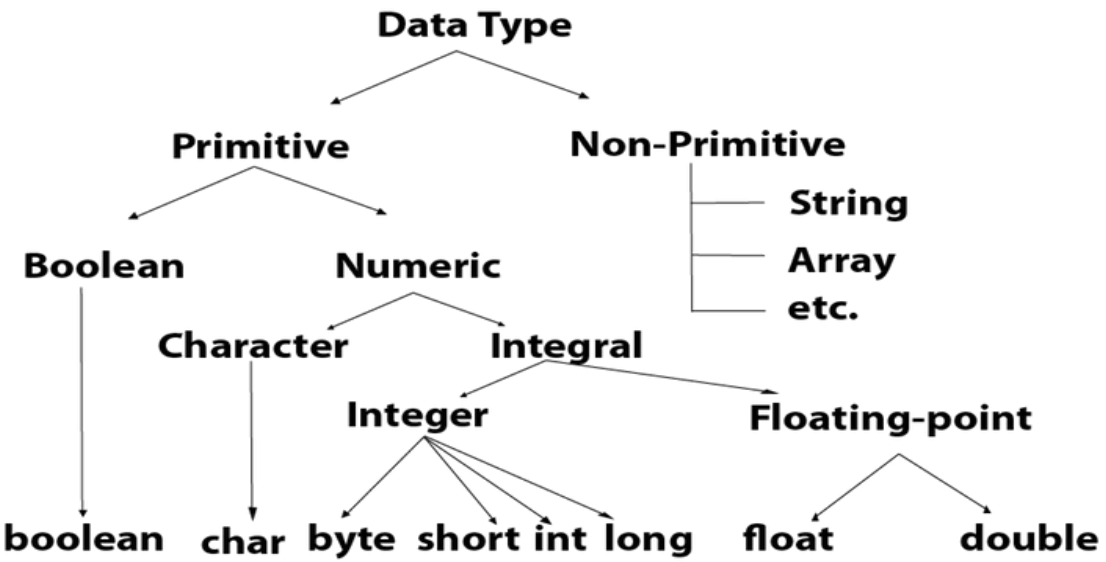
- In Java, identifiers are used for identification purposes. Java Identifiers can be a class name, method name, variable name, or label.
- **Rules For Defining Java Identifiers**
  - The only allowed characters for identifiers are all alphanumeric characters([**A-Z**],[**a-z**],[**0-9**]), ‘**$**‘(dollar sign) and ‘**_**‘ (underscore).For example “NIT@” is not a valid Java identifier as it contains a ‘@’ a special character.
  - Identifiers should **not** start with digits(**[0-9]**). For example “123geeks” is not a valid Java identifier.
  - Java identifiers are **case-sensitive**.
  - There is no limit on the length of the identifier but it is advisable to use an optimum length of 4 – 15 letters only.

Constants

- Numeric constants:
  - Integer : e.g. 132 , -992 , 12 etc.
  - Real : 3.4 , 56.88, -9.0
    - Read constants may be represented in exponential notation. E.g., 215.65 => 2.1565e2 (e2 mean 10^2)
    - General form: mantissa e exponent [mantissa : either a real numer of integer]
    - E.g.: 0.65e4, 12e-2, 1.5e+5, 3.18E3
- Character constants: ‘A’, ‘1’, ‘h’
- String constants: “abc”, “34” , “a”
- Symbolic constants: final int passing_mark=40; final float PI=3.1459;

Data types

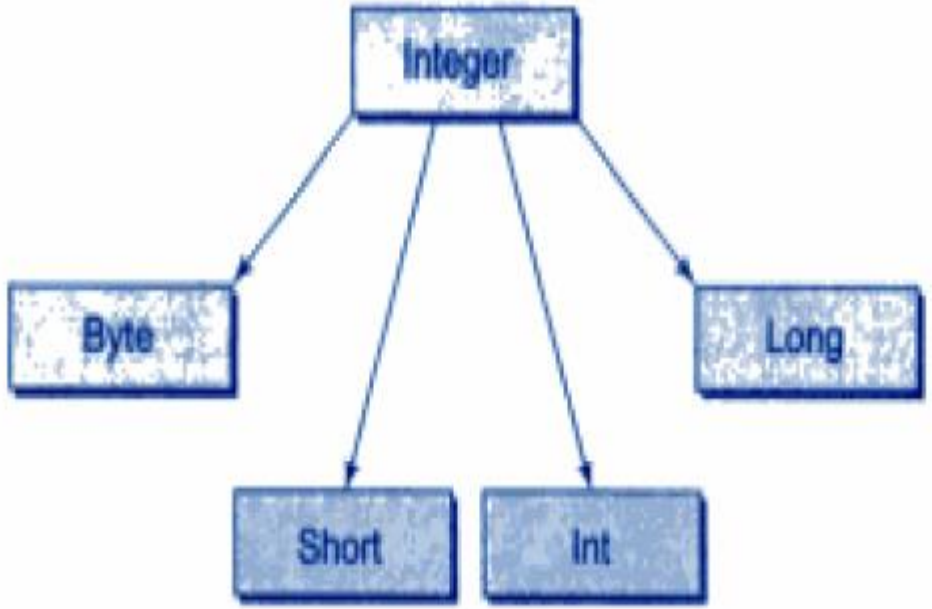
- Data Types in Java
- Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:
- **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
- **Non-primitive data types:** The non-primitive data types include Arrays, String etc.



Data types

Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Integer type



Type	*Size*	*Minimum value*	*Maximum value*
byte	One byte	–128	127
short	Two bytes	–32, 768	32, 767
int	Four bytes	–2, 147, 483, 648	2, 147, 483, 647
long	Eight bytes	–9, 223, 372, 036, 854, 775, 808	9, 223, 372, 036, 854, 775, 807

Floating point type

- Two datatypes to store floating point numbers.: float and double
- Float type values are single precision.
- Double type values are double precision.

Type	*Size*	*Minimum value*	*Maximum value*
float	4 bytes	3.4e-038	3.4e+038
double	8 bytes	1.7e-308	1.7e+308

Char data type

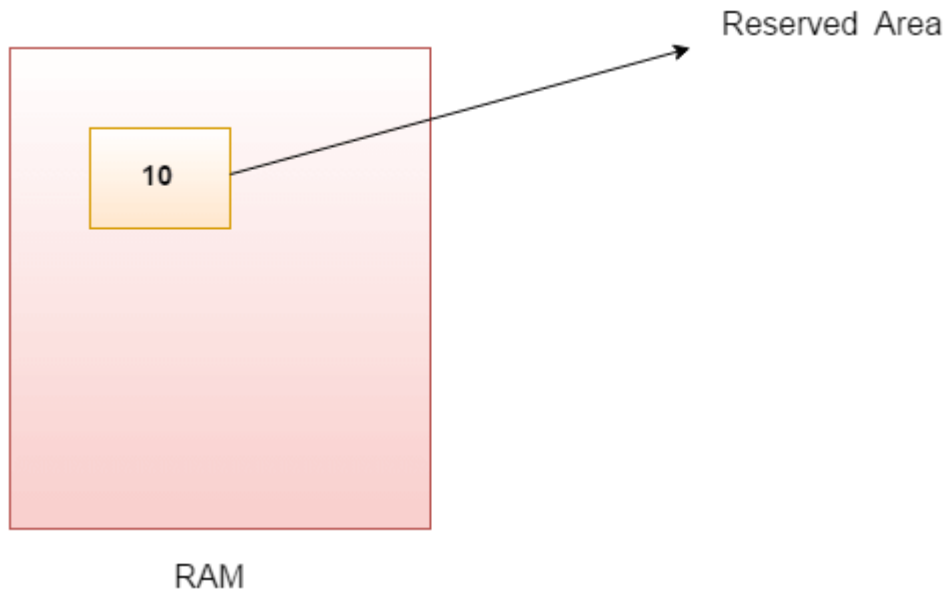
- The char data type is a single 16-bit Unicode character. Its value-range lies between 0 to 65,535 inclusive.
- **Why is the Size of char 2 bytes in Java?**
  - So, other languages like C/C++ use only ASCII characters, and to represent all ASCII characters 8 bits is enough. But Java uses the **Unicode system not the ASCII code System** and to represent the Unicode system 8 bits is not enough to represent all characters so Java uses 2 bytes for characters. **Unicode** defines a fully international character set that can represent most of the world's written languages.

Boolean type

- The Boolean data type is used to store only two possible values: true and false. This data type is used for simple flags that track true/false conditions.
- The Boolean data type specifies one bit of information, but its "size" can't be defined precisely.
- Boolean one = **false**
- Outcome of relational operator is Boolean.

Variables

- A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.
- Variable is a name of memory location.
- There are three types of variables in java: local, instance and static.
- **int** data=50;//Here data is variable



Local variable

- A variable defined within a block or method or constructor is called a local variable.
- The Local variable is created at the time of declaration and destroyed after exiting from the block or when the call returns from the function.
- The scope of these variables exists only within the block in which the variables are declared.
- Initialization of the local variable is mandatory before using it in the defined scope

```
class GFG {
public static void main(String[] args)
    {
        // Declared a Local Variable
        int var = 10;


        // This variable is local to this
main method only
        System.out.println("Local
Variable: " + var);
    }
}
```

Instance Variables

- Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block.
- As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
- Initialization of an instance variable is not mandatory. Its default value is dependent on the data type of variable. For *String* it is *null,* for *float* it is *0.0f,* for *int* it is *0,* for Wrapper classes like *Integer* it is *null, etc.*
- Instance variables can be accessed only by creating objects.

Instance variable

```
public class SumNumbers {
    // Instance variables
    int number1;
    int number2;
    // Method to calculate the sum of the instance variables
    public int calculateSum() {
        return number1 + number2;
    }
    public static void main(String[] args) {
        // Create an object of the SumNumbers class
        SumNumbers sumObj = new SumNumbers();
        // Assign values to instance variables
        sumObj.number1 = 10;
        sumObj.number2 = 20;
        // Calculate the sum and print it
        int result = sumObj.calculateSum();
        System.out.println("The sum of number1 and number2 is: " + result);
    }
}
```

Static variable

- These variables are declared similarly to instance variables. The difference is that static variables are declared using the static keyword within a class outside of any method, constructor, or block.
- Unlike instance variables, we can only have one copy of a static variable per class, irrespective of how many objects we create.
- Static variables are created at the start of program execution and destroyed automatically when execution ends.
- Default values of static variable depends on data type and same as instance variable.
- Access: className.variableName
- Static variables cannot be declared locally inside an instance method.

Static variable

```
Class A{
Static int count=5;
}
Class B{
public static void main(String[] args){
System.out.println(“Value of static variable : ” +A.count);
A.count+=10;
System.out.println(“Value of static variable : ” +A.count);
}
}
```

Scope of variable

- The **scope** of a variable refers to the part of the program where the variable is accessible/visible.
- A variable within a block or method or constructor are visible within that block or method or constructor .
- Instance variables are associated with an instance of the class and can be accessed by any method within the class.
- Static variables belong to the class rather than any particular instance and can be accessed by all instances of the class.

Lifetime of variable

- The **lifetime** of a variable refers to the duration for which the variable exists in memory during program execution.
- It is closely related to the scope but specifically deals with when the variable is created and destroyed.
- **Local Variables:**
- Lifetime: The lifetime of local variables begins when the method or block in which they are declared is executed, and ends when the method or block finishes execution.
- Memory: They are stored on the stack, and their memory is automatically deallocated when the method or block exits.
- **Instance Variables:**
- Lifetime: The lifetime of instance variables is tied to the lifetime of the object they belong to. They are created when the object is instantiated and destroyed when the object is garbage-collected.
- Memory: They are stored in the heap memory.
- **Static Variables:**
- Lifetime: The lifetime of static variables starts when the class is loaded into memory and ends when the class is unloaded, typically when the program terminates.
- Memory: They are stored in the static memory area of the heap.
- **Block Variables:**
- Lifetime: The lifetime of block-scoped variables is limited to the execution of the block in which they are declared.
- Memory: They exist in memory only for the duration of the block's execution.

Java program to take input

```
import java.util.Scanner;  // Import the Scanner class
public class InputExample {
    public static void main(String[] args) {
        // Create a Scanner object to take input from the user
        Scanner scanner = new Scanner(System.in);
     System.out.print("Enter your name: ");
        String name = scanner.nextLine();  // Read a line of text (String)
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();  // Read an integer
        System.out.print("Enter your height in meters: ");
double height = scanner.nextDouble();  // Read a double
        // Display the input received
        System.out.println("\nYour Details:");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Height: " + height + " meters");
        // Close the scanner to prevent resource leaks
        scanner.close();
    }
}
```

Different Functions of Scanner Class for Different Input Types

Method	Description
nextBoolean()	Reads a boolean value from the user
nextByte()	Reads a byte value from the user
nextDouble()	Reads a double value from the user
nextFloat()	Reads a float value from the user
nextInt()	Reads a int value from the user
nextLine()	Reads a String value from the user
nextLong()	Reads a long value from the user
nextShort()	Reads a short value from the user

Operators

- Operators in Java are the symbols used for performing specific operations in Java. Operators make tasks like addition, multiplication, etc which look easy although the implementation of these tasks is quite complex.
- **Types of Operators in Java**
- There are multiple types of operators in Java all are mentioned below:
  - Arithmetic Operators : * , / , % , + , –
  - Unary Operators
  - Assignment Operator
  - Relational Operators
  - Logical Operators
  - Ternary Operator
  - Bitwise Operators
  - Shift Operators
  - instanceof operator

Unary operators

- **– : Unary minus**
- **+ : Unary plus**
- **++ : Increment operator**, used for incrementing the value by 1.
  - **Post-Increment:** Value is first used for computing the result and then incremented.
  - **Pre-Increment:** Value is incremented first, and then the result is computed.
- **– – : Decrement operator**, used for decrementing the value by 1.
  - **Post-decrement:** Value is first used for computing the result and then decremented.
  - **Pre-Decrement: The value** is decremented first, and then the result is computed.
- **! : Logical not operator**, used for inverting a boolean value.

```
class Test {
    // main function
    public static void main(String[] args)
    {
        // Interger declared
        int a = 10;
        int b = 10;

        // Using unary operators
        System.out.println("Postincrement : " + (a++));
        System.out.println("Preincrement : " + (++a));

        System.out.println("Postdecrement : " + (b--));
        System.out.println("Predecrement : " + (--b));
    }
}
```

Assignment Operator

- **‘=’** Assignment operator is used to assign a value to any variable.
- It has right-to-left associativity.
- Assignment operator can be combined with other operators to build a shorter version of the statement called a **Compound Statement.**
- **+=**
- **-=**
- ***=**
- **/=**
- **%=**

Relational operator

- These operators are used to check for relations like equality, greater than, and less than.
- They return boolean results.
- **==, Equal to**
- **!=, Not Equal to**
- **<, less than**
- **<=, less than or equal to**
- **>, Greater than**
- **>=, Greater than or equal to**

```
class Test {
    // main function
    public static void main(String[] args)
    {
        // Comparison operators
        int a = 10;
        int b = 3;
        int c = 5;
        System.out.println("a > b: " + (a > b));
        System.out.println("a < b: " + (a < b));
        System.out.println("a >= b: " + (a >= b));
        System.out.println("a <= b: " + (a <= b));
        System.out.println("a == c: " + (a == c));
        System.out.println("a != c: " + (a != c));
    }
}
```

Logical operators

- These operators are used to perform “logical AND” and “logical OR” operations.
- One thing to keep in mind is the second condition is not evaluated if the first one is false.
- **&&, Logical AND:** returns true when both conditions are true.
- **||, Logical OR:** returns true if at least one condition is true.
- **!, Logical NOT:** returns true when a condition is false and vice-versa

```
class GFG {
    // Main Function
    public static void main (String[] args) {
        // Logical operators
        boolean x = true;
        boolean y = false;
        System.out.println("x && y: " + (x && y));
        System.out.println("x || y: " + (x || y));
        System.out.println("!x: " + (!x));
    }
}
```

Ternary operator

- The ternary operator is a shorthand version of the if-else statement. It has three operands and hence the name Ternary.
- condition ? if true : if false
- Write a program to find greatest among three numbers using ternary operators.

Bitwise Operators

- These operators are used to perform the manipulation of individual bits of a number.
- **&, Bitwise AND operator:** returns bit by bit AND of input values.
- **|, Bitwise OR operator:** returns bit by bit OR of input values.
- **^, Bitwise XOR operator:** returns bit-by-bit XOR of input values.
- **~, Bitwise Complement Operator:** This is a unary operator which returns the one's complement representation of the input value, i.e., with all bits inverted.

Example

```
public class BitwiseExample {
    public static void main(String[] args) {
        int a = 5;  // 0101 in binary
        int b = 9;  // 1001 in binary
        // AND Operator
        int andResult = a & b;  // 0001 in binary, which is 1
        System.out.println("a & b: " + andResult);
        // OR Operator
        int orResult = a | b; // 1101 in binary, which is 13
        System.out.println("a | b: " + orResult);
        // XOR Operator
        int xorResult = a ^ b; // 1100 in binary, which is 12
        System.out.println("a ^ b: " + xorResult);
        // NOT Operator
        int notResult = ~a;  // 1010 in binary (two's complement), which is -6
        System.out.println("~a: " + notResult);
}
}
```

Shift operator

- These operators are used to shift the bits of a number left or right, thereby multiplying or dividing the number by two, respectively.
- They can be used when we have to multiply or divide a number by two. General format-

number shift_op number_of_places_to_shift;

- **<<, Left shift operator:** shifts the bits of the number to the left and fills 0 on voids left as a result. The left shift by n positions multiplies the number by 2^n .

```
int a = 5; // Binary: 00000000 00000000 00000000 00000101
int result = a << 2;          // Left shift by 2 positions (5 * 2^2 = 20)
System.out.println(result);       // Output: 20
```

- **>>, Signed Right shift operator:** A bitwise operation that shifts the bits of a number to the right while preserving its original sign.
  - If the number is positive (the most significant bit is 0), the vacated leftmost positions are filled with 0s.
  - If the number is negative (the most significant bit is 1), the vacated leftmost positions are filled with 1s.

```
int a = 20; // Binary: 00000000 00000000 00000000 00010100
int result = a >> 2; // Right shift by 2 positions (20 / 2^2 = 5)
System.out.println(result);  // Output: 5
```

- **>>>, Unsigned Right shift operator:** shifts the bits of the number to the right and fills 0 on voids left as a result. The leftmost bit is set to 0.

```
int a = -20; // Binary: 11111111 11111111 11111111 11101100 (in 32-bit representation)
int result = a >>> 2; // Unsigned right shift by 2 positions
System.out.println(result); // Output: 1073741819
```

instanceof operator

- The instance of the operator is used for type checking.
- It can be used to test if an object is an instance of a class, a subclass, or an interface.
- Returns true or false.
- General format-

object instanceof *class/subclass/interface*

```
class Animal {

}

class Dog extends Animal {

}

public class InstanceofExample {


    public static void main(String[] args) {

        Animal myAnimal = new Animal();

        Dog myDog = new Dog();

        // Checking if myDog is an instance of Dog

        System.out.println("myDog is instance of Dog: " + (myDog instanceof
Dog));

     System.out.println("myDog is instance of Animal: " + (myDog instanceof
Animal));

      System.out.println("myAnimal is instance of Dog: " + (myAnimal
instanceof Dog));

System.out.println("myAnimal is instance of Animal: " + (myAnimal
instanceof Animal));

    }

}
```

Precedence and Associativity of Java Operators

- **Operator precedence** determines the order in which operators are applied in expressions. Operators with higher precedence are evaluated before operators with lower precedence.
- **Operator associativity** determines the direction in which operators of the same precedence level are evaluated.

Operators	Associativity	Type
++ --	Right to left	Unary postfix
++ -- + - ! (type)	Right to left	Unary prefix
/ * %	Left to right	Multiplicative
+ -	Left to right	Additive
< <= > >=	Left to right	Relational
== !=	Left to right	Equality
&	Left to right	Boolean Logical AND
^	Left to right	Boolean Logical Exclusive OR
|	Left to right	Boolean Logical Inclusive OR
&&	Left to right	Conditional AND
||	Left to right	Conditional OR
?:	Right to left	Conditional
= += -= *= /= %=	Right to left	Assignment

Type casting/coversion

- Type casting is used to convert a variable from one data type to another.
- **Implicit casting (**Widening) occurs automatically when you convert a smaller data type to a larger one. This is safe because the larger data type can accommodate the values of the smaller data type without loss of information.

```
int myInt = 10;
double myDouble = myInt;  // Implicit casting: int to double
System.out.println(myDouble);  // Output: 10.0
```

Type casting/conversion

- **Explicit casting (**Narrowing) is required when you convert a larger data type to a smaller one. This is because there is a potential for data loss. You must manually perform this conversion using a cast operator.

```
double myDouble = 9.78;
int myInt = (int) myDouble;  // Explicit casting: double to int
System.out.println(myInt);  // Output: 9
```

Control Structure

- Control structures in Java are used to control the flow of execution in a program.
- Java provides three types of control flow statements.
  - Decision Making statements
    - if statements
    - switch statement
  - Loop statements
    - do while loop
    - while loop
    - for loop
    - for-each loop
  - Jump statements
    - break statement
    - continue statement

Decision making constructs

```
if (condition) {
    // code
} else if (condition2) {
    // code
} else {
    // code
}
```

- Syntax is almost the same as C, but Java conditions must explicitly be boolean.

Switch statement

```
switch (variable/exp.) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code
}
```

- Switch in both C and Java are almost similar. C only supports integers and characters, while Java supports String datatype as well.

Looping statements

- Working of for, while and do-while loops is similar to C.
- **for-each loop**:
  - Used exclusively to loop through elements in an array.

```
for (type var : array)
{
  statements using var;
}
```

```
import java.io.*;
class Easy
{
	public static void main(String[] args)
	{
		int ar[] = { 10, 50, 60, 80, 90 };
		for (int element : ar)
		    System.out.print(element + " ");
	}
}
```

- **Limitation:**
- Not appropriate when you want to modify the array
- Do not keep track of index.
- Only iterates forward over the array in single steps.
- performance overhead over simple iteration

Jump statements

- **break statement** is used to exit loops (for, while, do-while) and switch statements.
- break statement can also be used with **labels**, which is not available in C. This allows breaking out of nested loops or blocks.

```
outerLoop:
for (int i = 0; i < 10; i++) {
    for (int j = 0; j < 10; j++) {
        if (j == 5) {
            break outerLoop;  // breaks out of the outer loop
        }
    }
}
```

Jump statements

- **Continue statement** skips the remaining code in the current iteration and moves to the next iteration of the loop.
- Java allows using **labeled continue** to control flow in nested loops, which is not available in C.

```
outerLoop:
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        if (j == 2) {
            continue outerLoop;  // skip the current iteration of the outer loop
        }
            System.out.println("i: " + i + ", j: " + j);
    }
}
```